CSE 213 — Software Engineering

Structural Design Patterns — Decorator, Composite, Adapter (+ Facade), Bridge

Lesson notes · Exam approach · TF practice with solutions · Quick notes (IJ Mam's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — structural-pattern questions by year
Lesson 1 — Decorator — pp. 1–50; Starbuzz, network packet layers, java.io
Lesson 2 — Composite — pp. 51–60; menus, employee hierarchy, FinCorp reports
Lesson 3 — Adapter (object vs class) and Facade — pp. 61–95; ducks/turkeys, zip-code weather, Bird/ToyDuck
Lesson 4 — Bridge — pp. 96–111; shapes × colours, notifications × channels
Quick Notes — Structural Patterns — revision + key structures

NOTE Slide numbers refer to the PDF page numbers of the deck named under each lesson title. TF page numbers refer to 'CSE 213 All Past Papers.pdf' (70 pages); the newest paper (22 batch, labelled 2024-25 here) is a separate file, cited as '22 batch p.N'. Old course code: CSE 307. Page numbers refer to '1 - StructuralDesignPattern.pdf' (113 pages; many pages are animation build-ups of the same slide, so ranges are given) and '3 - Design Pattern Misc.pdf' (8 pages). All Java here was compiled and run with javac (sources in _build/java/l1).

Exam Map — What the TF Papers Ask

Session (paper)	Questions in this note file
2024-25 (CSE 213)	Q2(a) FinCorp hierarchical reports (city/state/country) with optional tax/investment/compliance sections — Composite + Decorator in Java (20) [L2]
2023-24 (CSE 213)	Q5(b) One class acting as both Adapter and Facade: example + class diagram (12) [L3]
2022-23 (CSE 307)	Q1(a) Temperature by city, client passes zip code — class diagram + code (15) [L3]
2021-22 (CSE 307)	Q2(a) 5-layer network model: each layer adds header (and trailer) to Packet — identify pattern, implement, UML (20) [L1]
2020-21 (CSE 307)	Q3(a) Employee hierarchy with direct/indirect subordinates (15) [L2]
2019-20 (CSE 307)	Q2(a) MD → Technical/Admin/Finance managers → developers/executives/officers — Composite, show all employees under any employee (20) [L2]
2018-19 (CSE 307)	Q4(b) Bird / ToyDuck Adapter — class diagram + code (18) [L3]
2016-17 (CSE 307)	Q5(d) Class adapter UML; pros/cons of object vs class adapter (6⅔) [L3]
2011-12 (CSE 307)	Q1(a) Implementation issues and consequences of Composite (14) [L2]
2010-11 (CSE 307)	Q4(d) A group can be a member of another group — pattern and extension (12⅔) [L2] · Q4(e) LibX replaced by LibY with different interfaces — pattern + class diagram (7) [L3]

EXAM FOCUS One structural-pattern design question in almost every paper: 2024-25 Composite + Decorator (20, ★★★), 2023-24 Adapter + Facade (12), 2022-23 Adapter (15), 2021-22 Decorator (20), 2020-21 and 2019-20 Composite (15, 20), 2018-19 Adapter (18). Learn the four structure diagrams and the 'cue words' table at the end of each lesson. Facade, Proxy and Flyweight are not in IJ Mam's slides; Facade is covered briefly because 2023-24 asked it.

Lesson 1 — Decorator

Pages 1–50 (1 - StructuralDesignPattern) · Misc p. 2

Structural patterns explain how to assemble objects and classes into larger structures while keeping these structures flexible and efficient (pp. 2–4).

1. The Starbuzz problem (pp. 5–21)

- Beverage (description, getDescription(), abstract cost()) with HouseBlend, DarkRoast, Decaf, Espresso; customers add condiments — steamed milk, soy, mocha, whip — or not.
- Subclass per combination (DarkRoastWithMochaAndWhip …) → **class explosion** (pp. 11–13).
- Boolean flags in Beverage (hasMilk(), setMocha() …) and cost() adds them (pp. 14–21). Problems: a new condiment changes the superclass; iced tea inherits condiments it cannot take; no double mocha.

2. Decorate instead (pp. 22–46)

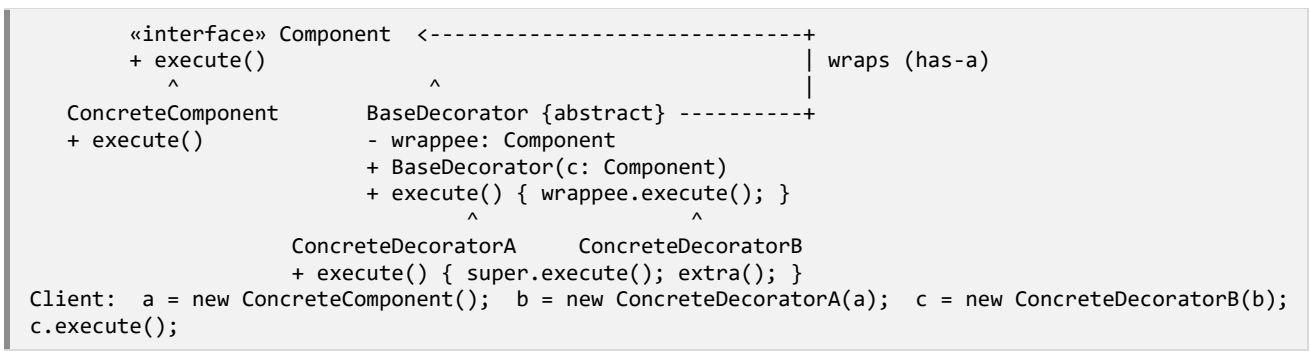
Start with a beverage and wrap it: DarkRoast inside Mocha inside Whip. cost() is delegated inward and each wrapper adds its own price: Whip.cost() = .10 + Mocha.cost() = .10 + .20 + DarkRoast.cost() (.99) = $1.29.

```
// Slides 33-46 – Decorator: Starbuzz beverages and condiments (Head First)
abstract class Beverage {
    String description = "Unknown Beverage";
    public String getDescription() { return description; }
    public abstract double cost();
}
abstract class CondimentDecorator extends Beverage {
    Beverage beverage;                                  // the wrapped component
    public abstract String getDescription();
}
class Espresso extends Beverage {
    Espresso() { description = "Espresso"; }
    public double cost() { return 1.99; }
}
class DarkRoast extends Beverage {
    DarkRoast() { description = "Dark Roast Coffee"; }
    public double cost() { return .99; }
}
class HouseBlend extends Beverage {
    HouseBlend() { description = "House Blend Coffee"; }
    public double cost() { return .89; }
}
class Mocha extends CondimentDecorator {
    Mocha(Beverage b) { beverage = b; }
    public String getDescription() { return beverage.getDescription() + ", Mocha"; }
    public double cost() { return .20 + beverage.cost(); }
}
class Whip extends CondimentDecorator {
    Whip(Beverage b) { beverage = b; }
    public String getDescription() { return beverage.getDescription() + ", Whip"; }
    public double cost() { return .10 + beverage.cost(); }
}
class Soy extends CondimentDecorator {
    Soy(Beverage b) { beverage = b; }
    public String getDescription() { return beverage.getDescription() + ", Soy"; }
    public double cost() { return .15 + beverage.cost(); }
}
public class StarbuzzCoffee {
    public static void main(String[] args) {
        Beverage b1 = new Espresso();
        System.out.printf("%s $%.2f%n", b1.getDescription(), b1.cost());
        Beverage b2 = new Whip(new Mocha(new Mocha(new DarkRoast())));
        System.out.printf("%s $%.2f%n", b2.getDescription(), b2.cost());
        Beverage b3 = new Whip(new Mocha(new Soy(new HouseBlend())));
        System.out.printf("%s $%.2f%n", b3.getDescription(), b3.cost());
    }
}
```

Output (as on p. 46): Espresso $1.99 · Dark Roast Coffee, Mocha, Mocha, Whip $1.49 · House Blend Coffee, Soy, Mocha, Whip $1.34.

3. Definition and structure (pp. 47–50)

MEMORIZE **Decorator** lets you attach new behaviours to objects by placing them inside special wrapper objects that contain the behaviours. (GoF: attach additional responsibilities to an object dynamically; a flexible alternative to subclassing.)



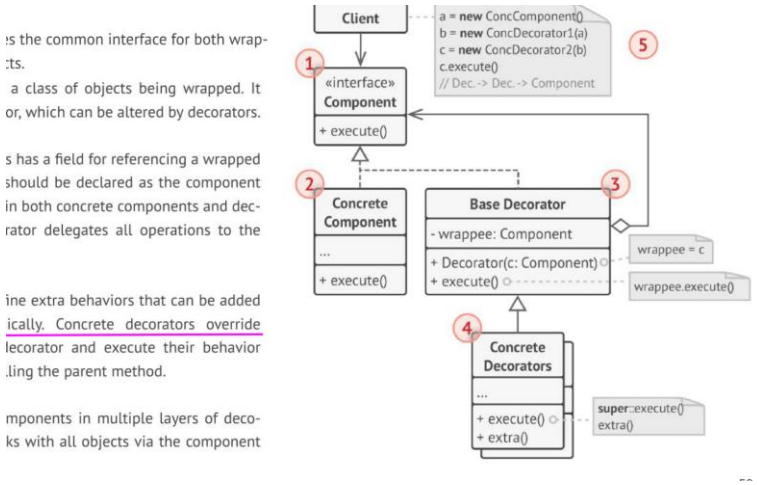


Figure 1.1 — Decorator structure (p. 50)

Participant	Role
Component	common interface of wrappers and wrapped objects
ConcreteComponent	the object being wrapped; basic behaviour
BaseDecorator	holds a reference of the Component type and delegates all work to it
ConcreteDecorators	add behaviour before/after calling the wrapped object
Client	wraps components in as many layers as needed, working through the Component interface

Real-world: java.io (Misc p. 2) — new BufferedReader(new InputStreamReader(new FileInputStream("data.txt"))): FileInputStream reads bytes (core), InputStreamReader adds byte→char conversion, BufferedReader adds buffering and readLine(). Open for extension, closed for modification.

Use Decorator when	Prefer instead
extras are optional, combinable, repeatable, chosen at run time	Strategy — to swap the whole algorithm, not stack layers
subclassing would explode or is impossible (final class)	Adapter — when the interface must change, not be kept
each layer adds one responsibility and keeps the interface	Composite — when objects form a part–whole tree

Practice — Lesson 1

(a) Actual — 2021-22 Q2(a).

(a) A five-layer network model: the user creates Packet(message, length) e.g. ('hello', 5); the Application layer adds APPHEADER_ (length 15), Transport TRHEADER_ (24), Network NETHEADER_ (34), Data-link DHEADER_ and _DTRAILER (51), Physical PHEADER_ (59) and returns the final Packet. Identify the pattern, implement it and give the UML class diagram. (20)

✓ Solution

(a) Decorator. Every layer has the same interface (send a packet), wraps the layer above it, and adds one header (data-link: header and trailer) without changing the others. Layers can be added, removed or re-ordered by changing the wrapping only. (Chain of Responsibility is the wrong answer: there each handler decides whether to pass the request on; here every layer always processes it.)

```
«interface» Layer + send(msg): Packet  <----------------------------+
   ^ RawPacket (creates Packet(msg))                                 | inner
   ^ LayerDecorator {abstract} - inner: Layer --------------------------+
       ^ ApplicationLayer  ^ TransportLayer  ^ NetworkLayer  ^ DataLinkLayer  ^ PhysicalLayer
Packet - message: String  + length(): int
```

```
// 2021-22 Q2(a) – Decorator: each layer wraps the packet and adds its header (data-link also a
trailer)
class Packet {
    final String message;
    Packet(String m) { message = m; }
    int length() { return message.length(); }
    public String toString() { return "message: " + message + ", length: " + length(); }
}
interface Layer { Packet send(String msg); }                // component

class RawPacket implements Layer {                           // concrete component: user's Packet
    public Packet send(String msg) { return new Packet(msg); }
}
abstract class LayerDecorator implements Layer {             // base decorator
    protected final Layer inner;
    LayerDecorator(Layer inner) { this.inner = inner; }
}
class ApplicationLayer extends LayerDecorator {
    ApplicationLayer(Layer l) { super(l); }
    public Packet send(String m) { return new Packet("APPHEADER_" + inner.send(m).message); }
}
class TransportLayer extends LayerDecorator {
    TransportLayer(Layer l) { super(l); }
    public Packet send(String m) { return new Packet("TRHEADER_" + inner.send(m).message); }
}
class NetworkLayer extends LayerDecorator {
    NetworkLayer(Layer l) { super(l); }
    public Packet send(String m) { return new Packet("NETHEADER_" + inner.send(m).message); }
}
class DataLinkLayer extends LayerDecorator {
    DataLinkLayer(Layer l) { super(l); }
    public Packet send(String m) { return new Packet("DHEADER_" + inner.send(m).message +
"_DTRAILER"); }
}
class PhysicalLayer extends LayerDecorator {
    PhysicalLayer(Layer l) { super(l); }
    public Packet send(String m) { return new Packet("PHEADER_" + inner.send(m).message); }
}
public class NetworkStack {
    public static void main(String[] args) {
        Layer stack = new PhysicalLayer(new DataLinkLayer(new NetworkLayer(
                      new TransportLayer(new ApplicationLayer(new RawPacket()))));
        System.out.println(new RawPacket().send("hello"));
        System.out.println(new ApplicationLayer(new RawPacket()).send("hello"));
        System.out.println(stack.send("hello"));
    }
}
```

Output: message: PHEADER_DHEADER_NETHEADER_TRHEADER_APPHEADER_hello_DTRAILER, length: 59 — the lengths 5, 15, 24, 34, 51, 59 match the question.

Lesson 2 — Composite

Pages 51–60 (1 - StructuralDesignPattern)

Pancake House and Diner menus are merged; the Waitress must print all menus, a submenu (desserts inside the diner menu) or the whole hierarchy (pp. 51–55). Menus contain menu items **and** other menus → a tree of nodes (Menu) and leaves (MenuItem) (p. 56).

```
Waitress --> «abstract» MenuComponent: getName(), getDescription(), getPrice(), isVegetarian(),
print(),
                                       add(c), remove(c), getChild(i)      (default: throw
UnsupportedOperation)
              ^ MenuItem (leaf): name, description, vegetarian, price; print()
              ^ Menu (composite): menuComponents: List<MenuComponent>; add, remove, getChild,
                                  print() { print own name; for each child: child.print(); }
```

MEMORIZE **Composite** lets you compose objects into tree structures and then work with these structures as if they were individual objects (p. 58).

```
Client --> «interface» Component + execute()  <-----------+
            ^ Leaf + execute()  (does the real work)      | children (0..*)
            ^ Composite - children: Component[] ----------+
                        + add(c), remove(c), getChildren()
                        + execute() { for each child: child.execute(); combine results }
```

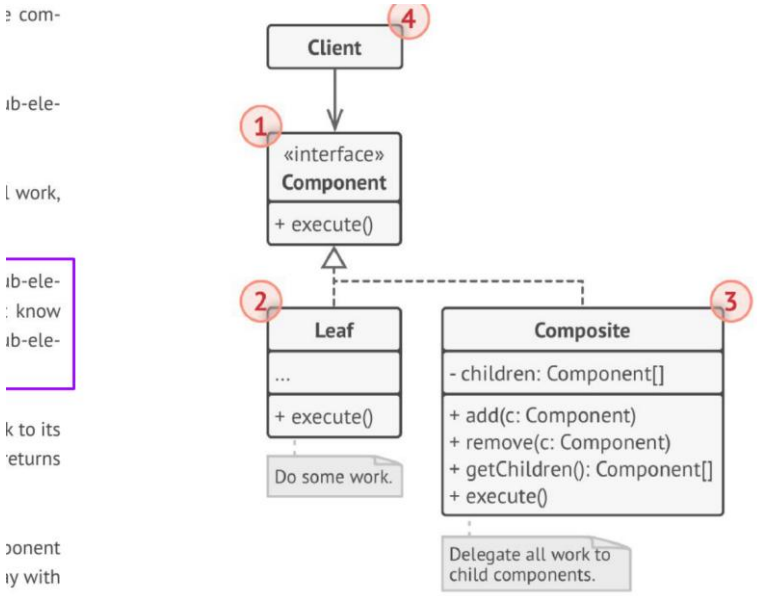


Figure 2.1 — Composite structure (p. 60)

Implementation issues and consequences (2011-12 Q1(a) — GoF background)

Issue	Choice
Where to declare add/remove	in Component (**transparency**: client treats all alike, but a leaf must reject add) or only in Composite (**safety**: compile-time check, but client must know the type)
Parent references	keep a parent link for upward navigation / deletion; maintain it in add/remove
Child ordering & storage	List if order matters (menus, layers); cache aggregated values if expensive
Who deletes children	the composite, when it is deleted (unless children are shared)
Sharing components	possible (Flyweight) but complicates parent references

Consequences: uniform treatment of leaves and composites; easy to add new component kinds; simple client code with recursion. Liability: the design can become overly general — hard to restrict which components a composite may hold.

Practice — Lesson 2

(a) Actual — 2019-20 Q2(a) / 2020-21 Q3(a). (b) Actual — 2024-25 Q2(a). (c) Actual — 2010-11 Q4(d). (d) Actual — 2011-12 Q1(a).

(a) A Managing Director heads Technical, Administration and Finance divisions, each with a Manager; five developers under the technical manager, three executives under admin, two officers under finance. Implement the hierarchy with Composite; any employee must be able to show all employees under him/her (direct and indirect). (20 / 15)

(b) FinCorp: offices in cities within states within countries; state, country and global reports aggregate their subordinates and attach level-specific analysis; optional sections (tax, investment, compliance) may be added in any combination without changing the core report. Identify the pattern(s) and implement minimally. (20)

(c) Extend the Facebook-group design (Observer) so a group page can itself be a member of another group ('CSE BUET 08' joins 'BUETIANS'). Which pattern, and how? (12⅔)

(d) Discuss the implementation issues and consequences of the Composite pattern. (14)

✓ Solution

(a) Component = Employee; leaf = Worker (developer, executive, officer); composite = Manager (MD and the three managers). showAll() prints the employee and recursively everyone below; countAll() counts direct + indirect.

```
// 2019-20 Q2(a), 2020-21 Q3(a) — Composite: employees and their (direct + indirect) subordinates
import java.util.ArrayList;
import java.util.List;

abstract class Employee {                                       // component
    protected final String name, designation;
    Employee(String n, String d) { name = n; designation = d; }
    void add(Employee e)    { throw new UnsupportedOperationException(); }
    void remove(Employee e) { throw new UnsupportedOperationException(); }
    List<Employee> getSubordinates() { return new ArrayList<>(); }
    abstract void showAll(String indent);                       // shows this employee and everyone
below
    public String toString() { return designation + ": " + name; }
}
class Worker extends Employee {                                 // leaf
    Worker(String n, String d) { super(n, d); }
    void showAll(String indent) { System.out.println(indent + this); }
}
class Manager extends Employee {                                // composite
    private final List<Employee> subordinates = new ArrayList<>();
    Manager(String n, String d) { super(n, d); }
    void add(Employee e)    { subordinates.add(e); }
    void remove(Employee e) { subordinates.remove(e); }
    List<Employee> getSubordinates() { return subordinates; }
    void showAll(String indent) {
        System.out.println(indent + this);
        for (Employee e : subordinates) e.showAll(indent + "   ");
    }
    int countAll() {                                           // direct + indirect
        int c = 0;
        for (Employee e : subordinates) c += 1 + (e instanceof Manager ? ((Manager) e).countAll() :
0);
        return c;
    }
}
public class CompanyHierarchy {
    public static void main(String[] args) {
        Manager md = new Manager("Karim", "Managing Director");
        Manager tech = new Manager("Rahim", "Technical Manager");
        Manager admin = new Manager("Salma", "Admin Manager");
        Manager fin = new Manager("Nabil", "Finance Manager");
        md.add(tech); md.add(admin); md.add(fin);
        for (int i = 1; i <= 5; i++) tech.add(new Worker("Dev" + i, "Developer"));
        for (int i = 1; i <= 3; i++) admin.add(new Worker("Exec" + i, "Executive"));
        for (int i = 1; i <= 2; i++) fin.add(new Worker("Officer" + i, "Officer"));
        md.showAll("");
        System.out.println("Under MD: " + md.countAll() + ", under Technical Manager: " +
tech.countAll());
    }
}
```

Output: the tree with MD at the top and 'Under MD: 13, under Technical Manager: 5'.

(b) Composite for requirement (i): CityOffice is a leaf; State, Country and Global are composites that sum their children's expenditure and run their own analysis(); all share the ReportUnit interface, so every stakeholder calls report() the same way. **Decorator** for requirement (ii): TaxSection, InvestmentSection and ComplianceSection wrap any ReportUnit and append their section, in any combination, without touching the core classes.

```
«interface» ReportUnit: name(), expenditure(), report()
   ^ CityOffice (leaf)
   ^ Region {abstract} (composite) - children: List<ReportUnit>; add(u); expenditure() = own + Σ children;
analysis()
        ^ State   ^ Country   ^ Global
   ^ ReportSection {abstract} (decorator) - inner: ReportUnit
        ^ TaxSection   ^ InvestmentSection   ^ ComplianceSection
```

```
// 2024-25 Q2(a) — Composite (city/state/country/global) + Decorator (optional report sections)
import java.util.ArrayList;
import java.util.List;

interface ReportUnit {                                  // the one consistent reporting interface
    String name();
    double expenditure();
    String report();
}
class CityOffice implements ReportUnit {                // leaf
    private final String name; private final double exp;
    CityOffice(String n, double e) { name = n; exp = e; }
    public String name() { return name; }
    public double expenditure() { return exp; }
    public String report() { return name + " expenditure = " + exp; }
}
abstract class Region implements ReportUnit {           // composite
    private final String name; private final double ownExpenditure;
    protected final List<ReportUnit> children = new ArrayList<>();
    Region(String n, double own) { name = n; ownExpenditure = own; }
    Region add(ReportUnit u) { children.add(u); return this; }
    public String name() { return name; }
    public double expenditure() {                       // aggregates subordinate units
        double s = ownExpenditure;
        for (ReportUnit u : children) s += u.expenditure();
        return s;
    }
    protected abstract String analysis();               // varies by level
    public String report() {
        return name + " total = " + expenditure() + " [" + analysis() + "]";
    }
}
class State extends Region {
    State(String n, double own) { super(n, own); }
    protected String analysis() {
        double max = 0; String top = "";
        for (ReportUnit u : children) if (u.expenditure() > max) { max = u.expenditure(); top = u.name(); }
        return "highest-spending city: " + top;
    }
}
class Country extends Region {
    Country(String n, double own) { super(n, own); }
    protected String analysis() { return "average per state: " + expenditure() / Math.max(1,
children.size()); }
}
```

```
class Global extends Region {
    Global(String n) { super(n, 0); }
    protected String analysis() {
        StringBuilder sb = new StringBuilder("share:");
        for (ReportUnit u : children) sb.append(String.format(" %s %.0f%%", u.name(), 100 * u.expenditure()
/ expenditure()));
        return sb.toString();
    }
}
abstract class ReportSection implements ReportUnit {   // decorator: same interface, wraps any unit
    protected final ReportUnit inner;
    ReportSection(ReportUnit u) { inner = u; }
    public String name() { return inner.name(); }
    public double expenditure() { return inner.expenditure(); }
}
class TaxSection extends ReportSection {
    TaxSection(ReportUnit u) { super(u); }
    public String report() { return inner.report() + "\n  + Tax details: 15% = " + 0.15 * expenditure(); }
}
class InvestmentSection extends ReportSection {
    InvestmentSection(ReportUnit u) { super(u); }
    public String report() { return inner.report() + "\n  + Investment summary attached"; }
}
class ComplianceSection extends ReportSection {
    ComplianceSection(ReportUnit u) { super(u); }
    public String report() { return inner.report() + "\n  + Government compliance report attached"; }
}
public class FinCorpReports {
    public static void main(String[] args) {
        State ny = (State) new State("New York", 0).add(new CityOffice("NYC", 500)).add(new
CityOffice("Buffalo", 200));
        State ca = (State) new State("California", 50).add(new CityOffice("LA", 400));
        Country usa = (Country) new Country("USA", 100).add(ny).add(ca);
        Country de = (Country) new Country("Germany", 0).add(new State("Bavaria", 0).add(new
CityOffice("Munich", 300)));
        Global world = (Global) new Global("FinCorp Global").add(usa).add(de);
        System.out.println(ny.report());                                    // State Director
        System.out.println(new TaxSection(usa).report());                   // Country Head + tax
        System.out.println(new ComplianceSection(new InvestmentSection(world)).report());  // CFO + 2
sections
    }
}
```

Output: New York total = 700 [highest-spending city: NYC] · USA total = 1250 + tax 187.5 · FinCorp Global total = 1550 [USA 81 %, Germany 19 %] + investment + compliance sections.

(c) Composite. Make the group itself a member: introduce GroupMember (Component) with receive(update); User is a leaf, Group is a composite that holds members (users or groups). When a post is published in BUETIANS, it calls receive() on each member; the 'CSE BUET 08' group forwards to all its own members, so they automatically become members of BUETIANS. Group keeps its Observer role (subject) and now also implements the observer/member interface.

```
«interface» GroupMember + receive(post: String)              (Observer + Component)
   ^ User (leaf): receive(p) { newsFeed.add(p); }
   ^ Group (composite + subject): members: List<GroupMember>; join(m); leave(m);
                                  publish(p) { for m in members: m.receive(p); }
                                  receive(p) { publish(p); }      // a member group re-broadcasts
```

(d) The issues table and consequences above.

Lesson 3 — Adapter (object vs class) and Facade

Pages 61–95 (1 - StructuralDesignPattern)

An adapter sits between your existing system and a vendor class with a different interface, like a plug adapter (pp. 61–65). Duck simulator (pp. 66–79): Duck (quack, fly), MallardDuck; vendor Turkey (gobble, fly — short distance), WildTurkey. TurkeyAdapter implements Duck: quack() → turkey.gobble(); fly() → five turkey.fly() calls.

```
public class TurkeyAdapter implements Duck {
    Turkey turkey;
    public TurkeyAdapter(Turkey turkey) { this.turkey = turkey; }
    public void quack() { turkey.gobble(); }
    public void fly() { for (int i = 0; i < 5; i++) turkey.fly(); }
}
```

MEMORIZE **Adapter** — Convert the interface of a class into another interface clients expect. Adapter lets classes work together that couldn't otherwise because of incompatible interfaces (p. 83).

```
OBJECT ADAPTER (composition, p. 86-88)            CLASS ADAPTER (inheritance, p. 89-90)
Client --> «interface» Target + request()          Client --> «interface» Target + request()
               ^                                                    ^
             Adapter - adaptee: Adaptee ---->  Adaptee          Adapter ------------|>  Adaptee
             + request() {                     + specificRequest()  + request() { specificRequest();
}
                 adaptee.specificRequest(); }                     (extends Adaptee AND implements
Target)
```

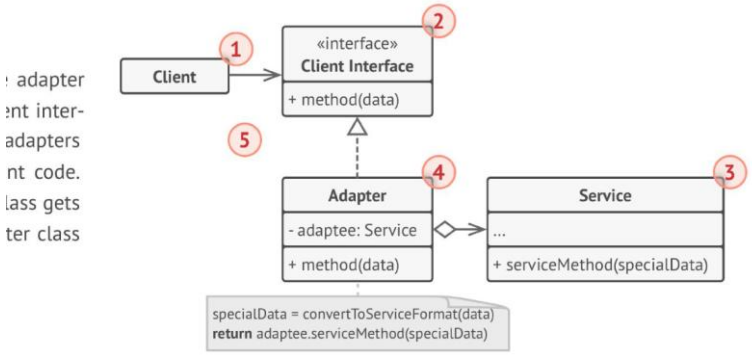


Figure 3.1 — Object adapter (p. 88)

	Object adapter	Class adapter
Mechanism	composition: holds an Adaptee	inheritance: subclasses Adaptee and implements Target
Language	works in every OO language	needs multiple inheritance (C++); in Java only if Target is an interface
Pros	adapts the Adaptee **and all its subclasses**; can be swapped at run time; keeps Adaptee hidden	no extra object; can **override** Adaptee behaviour; no need to re-implement the whole Adaptee
Cons	cannot override Adaptee methods easily; one more object/indirection	tied to one concrete Adaptee class; exposes Adaptee's public methods; inflexible

XML → JSON example (p. 85): a stock app downloads XML data but the analytics library wants JSON; an XML-to-JSON adapter converts the data before calling the library. Decorator vs Adapter (pp. 94–95): a decorator adds responsibilities and **keeps** the interface; an adapter **converts** the interface.

Facade (textbook — needed for 2023-24 Q5(b))

Facade provides a unified, simplified interface to a set of interfaces in a subsystem (home theater: watchMovie() turns on the projector, sound system and player in the right order). Adapter changes one interface to match a client; Facade simplifies many interfaces; Decorator adds behaviour. p. 82 notes an adapter may wrap more than one class — that is where Adapter and Facade meet.

Practice — Lesson 3

(a) Actual — 2022-23 Q1(a). (b) Actual — 2018-19 Q4(b). (c) Actual — 2023-24 Q5(b). (d) Actual — 2016-17 Q5(d). (e) Actual — 2010-11 Q4(e).

(a) A service returns the temperature for a city name; the client wants to pass a zip code. Class diagram + code. (15)

(b) Bird (fly(), makeSound()) and ToyDuck (squeak()): use Bird objects in place of ToyDucks. Class diagram + code. (18)

(c) Show how a single class can serve as both an Adapter and a Facade with an example; give the class diagram and analyse the dual role. (12)

(d) Draw the class-adapter UML; pros and cons of object adapter vs class adapter. (6⅔)

(e) The application uses LibX (vendor out of business); LibY offers the same functionality with different interfaces; no source code for either. Which pattern? Class diagram. (7)

✓ Solution

(a) Object Adapter. Target = ZipWeatherService.getTemperature(int zip); Adaptee = existing CityWeatherService.getTemperature(String city); Adapter maps the zip code to a city and delegates.

```
Client --> «interface» ZipWeatherService + getTemperature(zip: int): double
                ^
          ZipToCityAdapter - service: CityWeatherService - zipToCity: Map<int,String> --->
CityWeatherService
          + getTemperature(zip) { return service.getTemperature(zipToCity.get(zip)); }   +
getTemperature(city: String)
```

```
// 2022-23 Q1(a) — Object Adapter: clients pass a zip code; the existing service needs a city name
import java.util.HashMap;
import java.util.Map;

class CityWeatherService {                                    // adaptee (existing, unchanged)
    double getTemperature(String city) {
        switch (city) { case "Dhaka": return 31.5; case "Chittagong": return 29.0; default: return
Double.NaN; }
    }
}
interface ZipWeatherService { double getTemperature(int zipCode); }   // target expected by the
client

class ZipToCityAdapter implements ZipWeatherService {           // adapter
    private final CityWeatherService service;
    private final Map<Integer, String> zipToCity = new HashMap<>();
    ZipToCityAdapter(CityWeatherService s) {
        service = s;
        zipToCity.put(1000, "Dhaka");
        zipToCity.put(4000, "Chittagong");
    }
    public double getTemperature(int zipCode) {
        String city = zipToCity.get(zipCode);
        if (city == null) throw new IllegalArgumentException("Unknown zip " + zipCode);
        return service.getTemperature(city);                     // translate, then delegate
    }
}
public class WeatherAdapter {
    public static void main(String[] args) {
        ZipWeatherService ws = new ZipToCityAdapter(new CityWeatherService());
        System.out.println("1000 -> " + ws.getTemperature(1000) + " C");
        System.out.println("4000 -> " + ws.getTemperature(4000) + " C");
    }
}
```

(b) Target = ToyDuck; Adaptee = Bird; BirdAdapter implements ToyDuck and holds a Bird; squeak() → makeSound().

```
// 2018-19 Q4(b) — Adapter: use Bird objects where ToyDuck objects are expected
interface Bird { void fly(); void makeSound(); }
class Sparrow implements Bird {
    public void fly() { System.out.println("Flying"); }
    public void makeSound() { System.out.println("Chirp Chirp"); }
}
interface ToyDuck { void squeak(); }                          // target
class PlasticToyDuck implements ToyDuck {
    public void squeak() { System.out.println("Squeak"); }
}
class BirdAdapter implements ToyDuck {                        // adapter
    private final Bird bird;                                  // adaptee
    BirdAdapter(Bird bird) { this.bird = bird; }
    public void squeak() { bird.makeSound(); }                // translate the call
}
public class BirdAdapterDemo {
    public static void main(String[] args) {
        ToyDuck[] ducks = { new PlasticToyDuck(), new BirdAdapter(new Sparrow()) };
        for (ToyDuck d : ducks) d.squeak();                   // client sees only ToyDuck
    }
}
```

(c) SmartTVController implements the MediaPlayer interface the remote-control app expects. play(title) is a **Facade** method — one call that powers the projector, selects input, powers and sets the sound system — and at the same time an **Adapter** — it translates play(title) into the legacy streamer's incompatible startStream(contentCode, startSeconds). Analysis: the facade role reduces coupling to many subsystem classes; the adapter role resolves one interface mismatch; combining them is fine while the class stays small, but if it grows, split into a pure StreamerAdapter used by a HomeTheaterFacade (SRP).

```
RemoteApp --> «interface» MediaPlayer + play(title)
                  ^
            SmartTVController + play(title)  ---> Projector (on, input)        } facade over the subsystem
                                             ---> SoundSystem (powerUp, setVolume)
                                             ---> LegacyStreamer (startStream(code, secs))  } adaptee
```

```
// 2023-24 Q5(b) — one class acting as both Adapter and Facade
class Projector { void on() { System.out.println("Projector on"); } void input(String s) {
System.out.println("Projector input " + s); } }
class SoundSystem { void powerUp() { System.out.println("Sound up"); } void setVolume(int v) {
System.out.println("Volume " + v); } }
class LegacyStreamer {                                        // incompatible interface: wants seconds
and a code
    void startStream(String contentCode, int startSeconds) {
        System.out.println("Streaming " + contentCode + " from " + startSeconds + "s");
    }
}
interface MediaPlayer { void play(String title); }            // what the client (remote app) expects

class SmartTVController implements MediaPlayer {
    private final Projector projector = new Projector();
    private final SoundSystem sound = new SoundSystem();
    private final LegacyStreamer streamer = new LegacyStreamer();
    public void play(String title) {                          // FACADE: one call hides the subsystem
steps
        projector.on();
        projector.input("HDMI1");
        sound.powerUp();
        sound.setVolume(20);
        streamer.startStream(toCode(title), 0);               // ADAPTER: play(title) ->
startStream(code, secs)
    }
    private String toCode(String title) { return "VID-" + Math.abs(title.hashCode() % 10000); }
}
public class HomeTheater {
    public static void main(String[] args) { new SmartTVController().play("Inception"); }
}
```

(d) Class-adapter diagram in the box above (Adapter extends Adaptee, implements Target); pros/cons table.

(e) Adapter. Keep the application's code written against LibX's interface (extract it as an interface LibXApi if needed); write LibXToLibYAdapter classes that implement those interfaces and delegate each call to the corresponding LibY classes (object adapters, since we cannot modify either library).

```
Application --> «interface» LibXApi (the old method signatures)
                      ^
               LibXToLibYAdapter - y: LibYClass  ---> LibYClass (new, different interface)
               + oldMethod(args) { convert args; return y.newMethod(converted); }
```

Lesson 4 — Bridge

Pages 96–111 (1 - StructuralDesignPattern) · Misc pp. 5–8

Shapes (Circle, Square) × colours (Red, Blue): subclassing every combination (RedCircle, BlueSquare …) grows as shapes × colours (pp. 96–103). Split into two hierarchies — form and colour — and let Shape **contain** a Color: inheritance becomes composition (p. 104).

> **MEMORIZE** **Bridge** lets you split a large class or a set of closely related classes into two separate hierarchies — abstraction and implementation — which can be developed independently of each other (p. 108).

```
Client --> Abstraction  - i: Implementation  <>--------------> «interface» Implementation
           + feature1() { i.method1(); }                            + method1(), method2(), method3()
           + feature2()                                                ^                 ^
              ^                                                 ConcreteImplA     ConcreteImplB
        RefinedAbstraction + featureN()
```

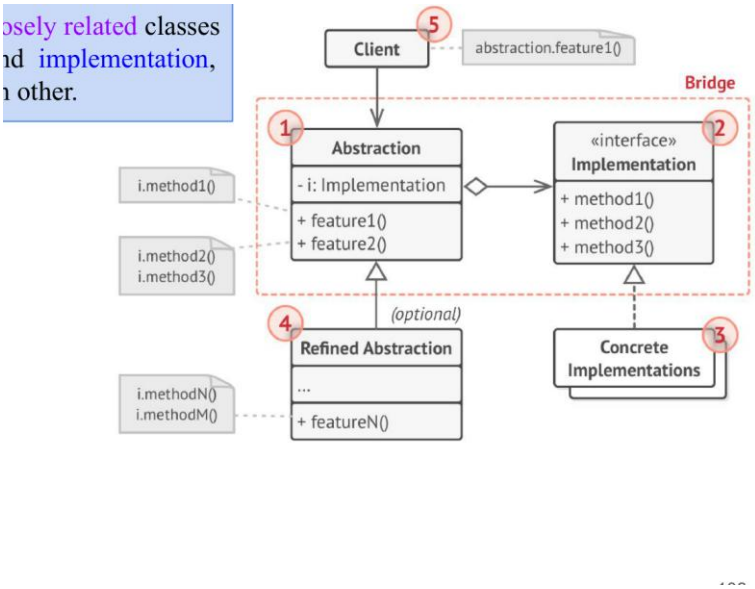


Figure 4.1 — Bridge structure (p. 108)

Abstraction = high-level control layer (GUI); it delegates the real work to the implementation layer (platform APIs: Linux, Windows, macOS) (pp. 105–107). Implementations are interchangeable; the same GUI works under different OSs.

Notification example (Misc pp. 7–8)

Abstractions: Alert, Promotion, OTP notifications; implementations: Email, SMS, Push. Without Bridge 3 × 3 = 9 classes; with Bridge 3 + 3 classes and any combination at run time.

```
// IJ Misc deck p7-8 — Bridge: notification kind x sending channel
interface MessageSender { void send(String msg); }                    // implementation hierarchy
class EmailSender implements MessageSender { public void send(String m) { System.out.println("Email:
" + m); } }
class SmsSender   implements MessageSender { public void send(String m) { System.out.println("SMS: "
+ m); } }
class PushSender  implements MessageSender { public void send(String m) { System.out.println("Push: "
+ m); } }

abstract class Notification {                                         // abstraction
    protected final MessageSender sender;                             // the bridge
    Notification(MessageSender s) { sender = s; }
    abstract void notifyUser();
}
class AlertNotification extends Notification {
    AlertNotification(MessageSender s) { super(s); }
    void notifyUser() { sender.send("Server is down!"); }
}
class OtpNotification extends Notification {
    OtpNotification(MessageSender s) { super(s); }
    void notifyUser() { sender.send("Your OTP is 482913"); }
}
class PromotionNotification extends Notification {
    PromotionNotification(MessageSender s) { super(s); }
    void notifyUser() { sender.send("50% off this weekend"); }
}
public class NotificationBridge {
    public static void main(String[] args) {
        new AlertNotification(new SmsSender()).notifyUser();
        new OtpNotification(new EmailSender()).notifyUser();
        new PromotionNotification(new PushSender()).notifyUser();
    }
}
```

Pattern	Intent in one line	Cue words
Decorator	add responsibilities by wrapping; same interface	optional add-ons, layers, extra cheese, headers
Composite	part–whole trees treated uniformly	hierarchy, sub-menu, subordinates, group in a group, aggregate totals
Adapter	convert one interface to the one clients expect	existing/vendor class, incompatible, zip vs city, LibX → LibY
Facade	one simple interface to a subsystem	many steps hidden behind one call
Bridge	abstraction and implementation vary independently	two dimensions, platform, channel, colour × shape

Practice — Lesson 4

(a) Constructed (Bridge vs Adapter). (b) Constructed.

(a) Bridge and Adapter both delegate to another object. What is the difference in intent and timing?

(b) A drawing app draws Circle and Rectangle using either a vector or a raster renderer. Give the Bridge diagram.

✓ Solution

(a) Bridge is designed **up front** to let two dimensions vary independently (abstraction ↔ implementation chosen at run time). Adapter is applied **after the fact** to make an existing, incompatible class fit an expected interface.

```
Shape {abstract} - renderer: Renderer ------> «interface» Renderer + drawCircle(x,y,r),
drawRect(x,y,w,h)
  + draw()                                          ^ VectorRenderer     ^ RasterRenderer
  ^ Circle  draw() { renderer.drawCircle(...) }
  ^ Rectangle draw() { renderer.drawRect(...) }
```

Quick Notes — Structural Patterns

Revision points for Lessons 1–4.

L1 · Decorator

- Wrapper with the same interface; cost/behaviour delegated inward + own addition.
- Starbuzz: DarkRoast + Mocha + Mocha + Whip = $1.49. java.io BufferedReader.
- Network packet layers = Decorator (each always adds a header).

L2 · Composite

- Component, Leaf, Composite(children); operation recurses over children.
- Transparency vs safety of add/remove; parent links; ordering.
- Employee hierarchy, FinCorp (Composite + Decorator), group inside group.

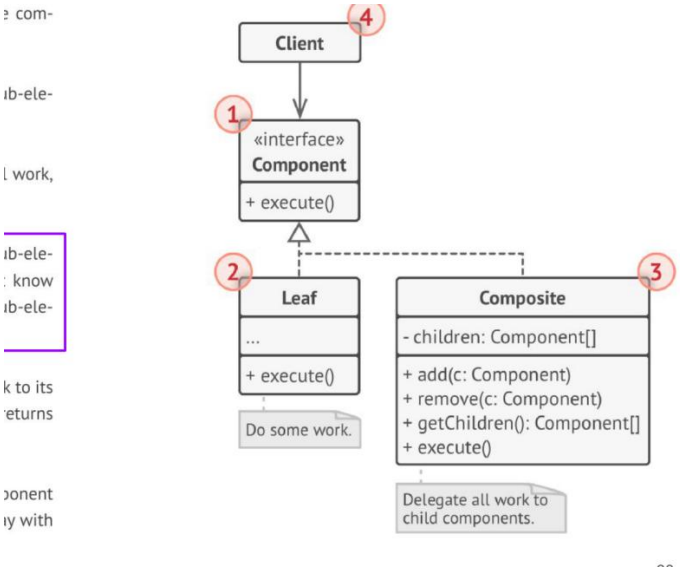
L3 · Adapter / Facade

- Target, Adapter, Adaptee; object (composition) vs class (inheritance).
- Zip → city, Bird → ToyDuck, Turkey → Duck, LibX → LibY.
- Facade simplifies many interfaces; Adapter converts one.

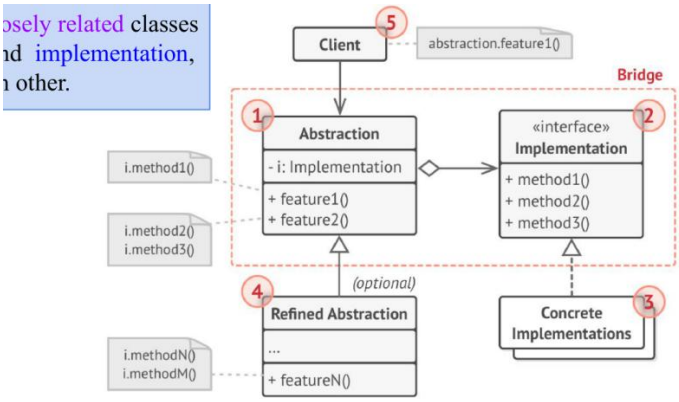
L4 · Bridge

- Abstraction has an Implementation; both hierarchies vary independently.
- Shapes × colours; notifications × channels (9 → 6 classes).

Key Diagrams to Reproduce



Composite (p. 60)


osely related classes
nd implementation,
other.
Client
5
abstraction.feature1()
Bridge
1
Abstraction
- i: Implementation
+ feature1()
+ feature2()
2
«interface»
Implementation
+ method1()
+ method2()
+ method3()
i.method1()
i.method2()
i.method3()
(optional)
4
Refined Abstraction
...
+ featureN()
3
Concrete
Implementations
i.methodN()
i.methodM()

Bridge (p. 108)